

Neural Data Science

Lecturer: Dr. Jan Lause, Prof. Dr. Philipp Berens

Tutors: Jonas Beck, Fabio Seel, Julius Würzler

Summer term 2025

Student names: Maria Elisabeth Dirnberger, Akseli Ilmanen, Caroline Griebhaber

LLM Disclaimer: *Did you use an LLM to solve this exercise? If yes, which one and where did you use it? [Copilot, Claude, ChatGPT, etc.]*

Coding Lab 4

In this notebook you will work with preprocessed 2 photon calcium recordings, that have already been converted into spike counts for a population of cells from the Macaque V1. During the experiment the animal has been presented with several drifting grating stimuli, in response to which the neural activity was recorded. In this exercise sheet we will study how you can visualize the activity of multiple neural spike trains and assess whether a neuron is selective to a specific stimulus type.

Download the data files `nds_cl_4_*.csv` from ILIAS and save it in the subfolder `../data/`. We recommend you to use a subset of the data for testing and debugging, ideally focus on a single cell (e.g. cell number x). The spike times and stimulus conditions are read in as pandas data frames. You can solve the exercise by making heavy use of that, allowing for many quite compact computations. See [documentation](#) and several good [tutorials](#) on how to do this. Of course, converting the data into classical numpy arrays is also valid.

```
In [1]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np
import scipy.optimize as opt

from scipy import signal as signal
from typing import Tuple

import itertools

%matplotlib inline

%load_ext jupyter_black
```

```
%load_ext watermark
%watermark --time --date --timezone --updated --python --iversions --waterma
```

Last updated: 2025-05-18 21:52:15W. Europe Summer Time

Python implementation: CPython
Python version : 3.10.0
IPython version : 8.35.0

sklearn: 1.6.1

numpy : 2.2.4
seaborn : 0.13.2
scipy : 1.15.2
matplotlib: 3.10.1
pandas : 2.2.3

Watermark: 2.5.0

```
In [2]: plt.style.use("../matplotlib_style.txt")
```

Load data

```
In [3]: spikes = pd.read_csv("../data/nds_cl_4_spiketimes.csv") # neuron id, spike
stims = pd.read_csv("../data/nds_cl_4_stimulus.csv") # stimulus onset in ms

stimDur = 2000.0 # in ms
nTrials = 11 # number of trials per condition
nDirs = 16 # number of conditions
deltaDir = 22.5 # difference between conditions

stims["StimOffset"] = stims["StimOnset"] + stimDur
```

```
In [4]: stims
```

Out[4]:

	StimOnset	Dir	StimOffset
0	15570	270.0	17570.0
1	19022	45.0	21022.0
2	22592	112.5	24592.0
3	26095	225.0	28095.0
4	29431	180.0	31431.0
...	...	...	...
171	583583	315.0	585583.0
172	586836	22.5	588836.0
173	590355	67.5	592355.0
174	593641	180.0	595641.0
175	597177	202.5	599177.0

176 rows × 3 columns

In []:

We require some more information about the spikes for the plots and analyses we intend to make later. With a solution based on dataframes, it is natural to compute this information here and add it as additional columns to the `spikes` dataframe by combining it with the `stims` dataframe. We later need to know which condition (`Dir`) and trial (`Trial`) a spike was recorded in, the relative spike times compared to stimulus onset of the stimulus it was recorded in (`relTime`) and whether a spike was during the stimulation period (`stimPeriod`). But there are many options how to solve this exercise and you are free to choose any of them.

```
In [5]: # you may add computations as specified above
spikes["Dir"] = np.nan
spikes["relTime"] = np.nan
spikes["Trial"] = np.nan
spikes["stimPeriod"] = np.nan

dirs = np.unique(stims["Dir"])
trialcounter = np.zeros_like(dirs)

for i, row in stims.iterrows():
    trialcounter[dirs == row["Dir"]] += 1

    i0 = spikes["SpikeTimes"] > row["StimOnset"]
    i1 = spikes["SpikeTimes"] < row["StimOffset"]

    select = i0.values & i1.values
```

```

spikes.loc[select, "Dir"] = row["Dir"]
spikes.loc[select, "Trial"] = trialcounter[dirs == row["Dir"]][0]
spikes.loc[select, "relTime"] = spikes.loc[select, "SpikeTimes"] - row["Dir"]
spikes.loc[select, "stimPeriod"] = True

spikes = spikes.dropna()

```

C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\4012369867.py:21: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'True' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```

spikes.loc[select, "stimPeriod"] = True

```

In [6]: `spikes.head()`

Out[6]:

	Neuron	SpikeTimes	Dir	relTime	Trial	stimPeriod
514	1	15739.000000	270.0	169.000000	1.0	True
515	1	15776.566667	270.0	206.566667	1.0	True
516	1	15808.466667	270.0	238.466667	1.0	True
517	1	15821.900000	270.0	251.900000	1.0	True
518	1	15842.966667	270.0	272.966667	1.0	True

Task 1: Plot spike rasters

In a raster plot, each spike is shown by a small tick at the time it occurs relative to stimulus onset. Implement a function `plotRaster()` that plots the spikes of one cell as one trial per row, sorted by conditions (similar to what you saw in the lecture). Why are there no spikes in some conditions and many in others?

If you opt for a solution without a dataframe, you need to change the interface of the function.

Grading: 3 pts

In [7]:

```

def plotRaster(spikes: pd.DataFrame, neuron: int):
    """plot spike rasters for a single neuron sorted by condition

    Parameters
    -----
    spikes: pd.DataFrame
        Pandas DataFrame with columns
            Neuron | SpikeTimes | Dir | relTime | Trial | stimPeriod

    neuron: int
        Neuron ID

    Note

```

```

----

this function does not return anything, it just creates a plot!
"""

# -----
# Write a raster plot function for the data (2 pts)
# -----

# Create a subset of the dataframe
# only comprising information about the neuron of interest
spikes_n = spikes[spikes.Neuron == neuron]

# Create a numpy array with all directions for plotting
# and store the number of directions
directions = np.arange(0, 360, deltaDir)[::-1]
n_directions = directions.shape[0]

# Plot the spikes for each direction
fig, ax = plt.subplots(n_directions, 1, sharex=True, figsize=(9, 9))
for idx in range(n_directions):

    # Plot the spikes against the time
    spikes_d = spikes_n[spikes_n.Dir == directions[idx]]
    ax[idx].scatter(spikes_d.relTime, spikes_d.Trial, c="C0", s=5)

    ax[idx].set_ylim(0, nTrials + 1)
    ax[idx].set_yticks([])
    ax[idx].set_ylabel(str(directions[idx]))

ax[0].set_title(f"Spike rasters for neuron {neuron}")
ax[nDirs - 1].set_xlabel("Relative time (ms)")

# A little bit of manual help
ax[6].set_ylabel("Direction of the stimulus\n202.5")

plt.show()

pass

```

Find examples of

1. a direction selective neuron
2. an orientation selective neuron
3. neither

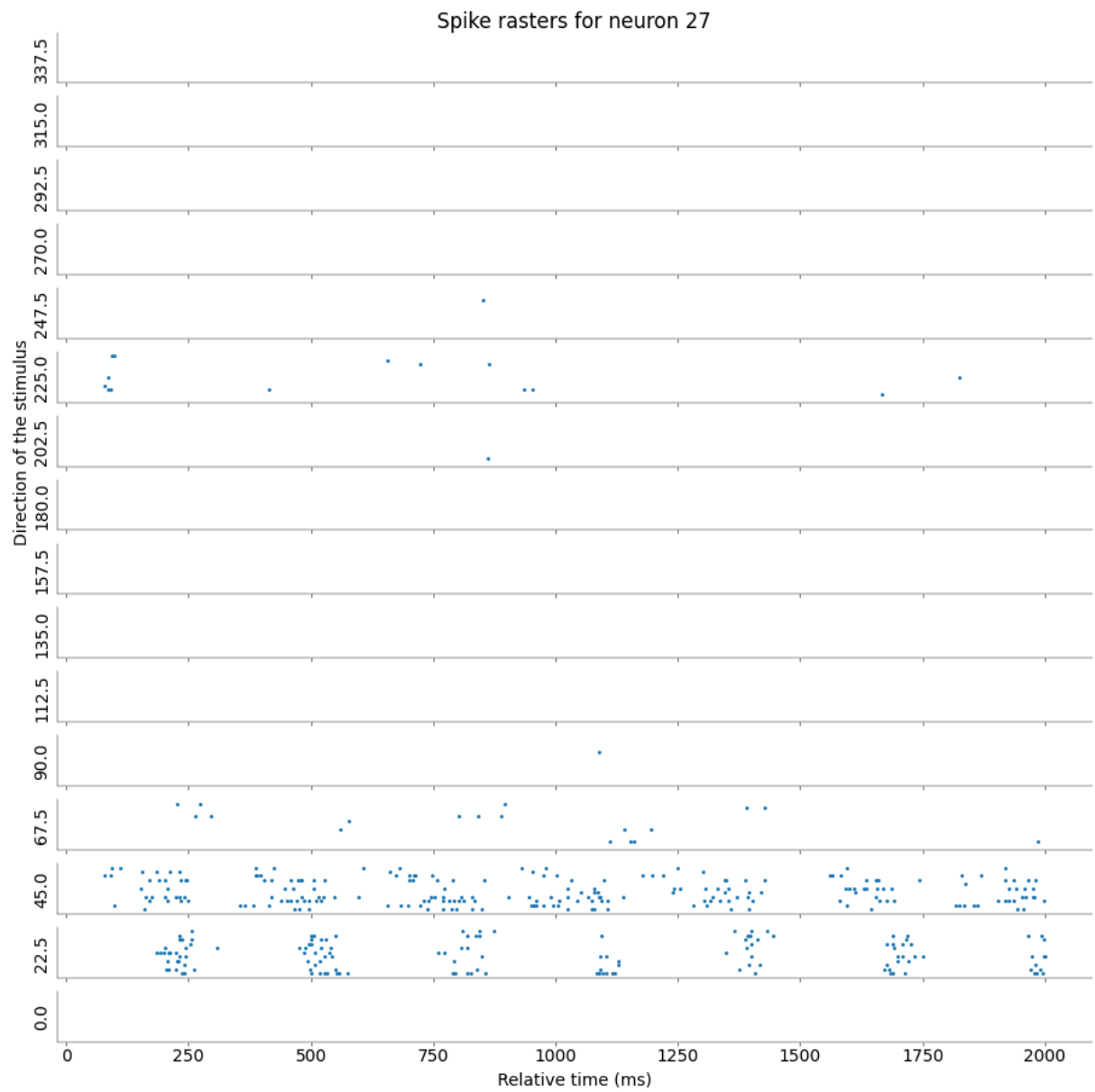
and explain your choices.

```

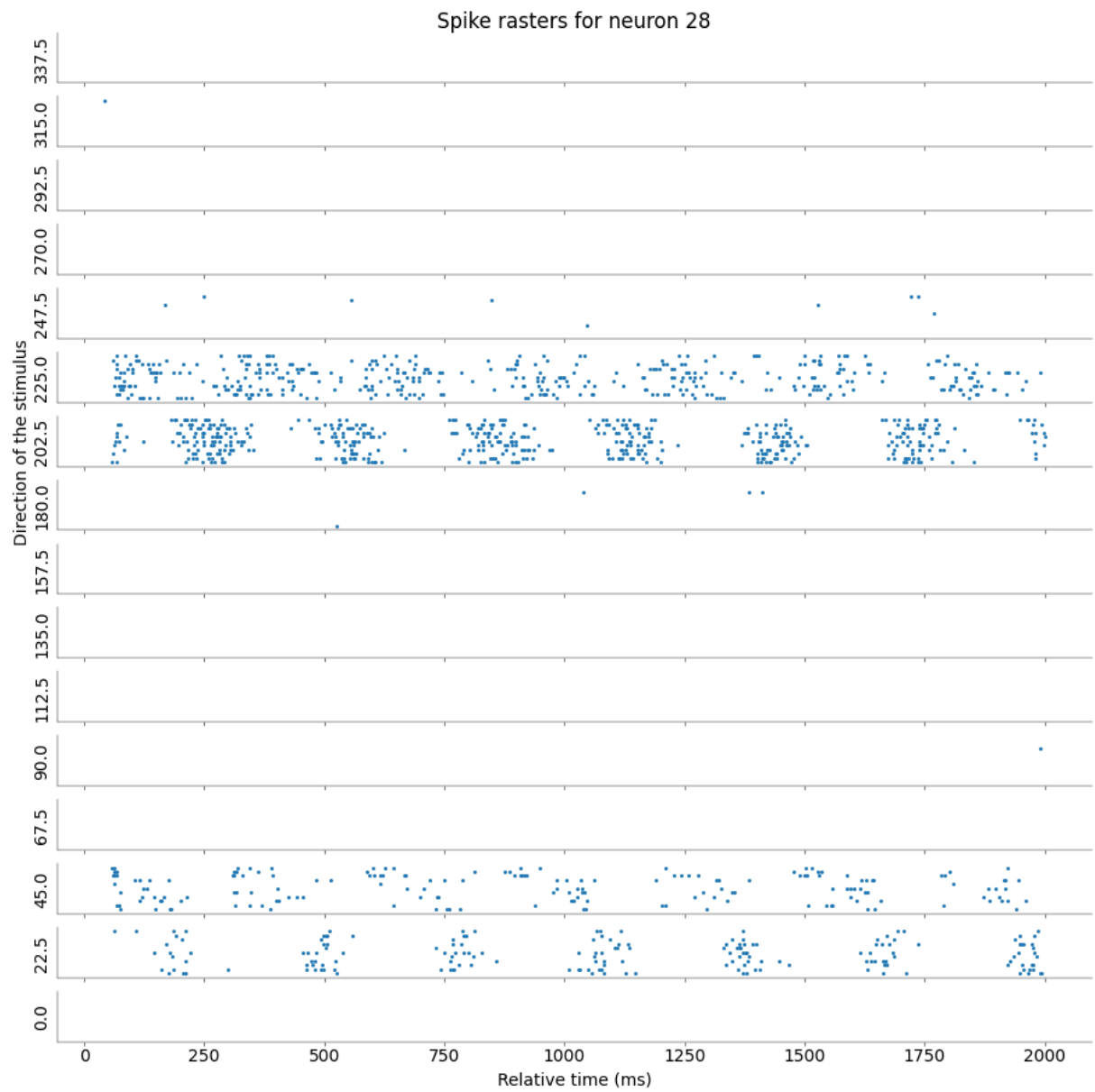
In [8]: # -----
# Find and explain examples? (1 pt)
# -----

# 1. direction selective: neuron responds more to motion in a specific direction
plotRaster(spikes, 27)

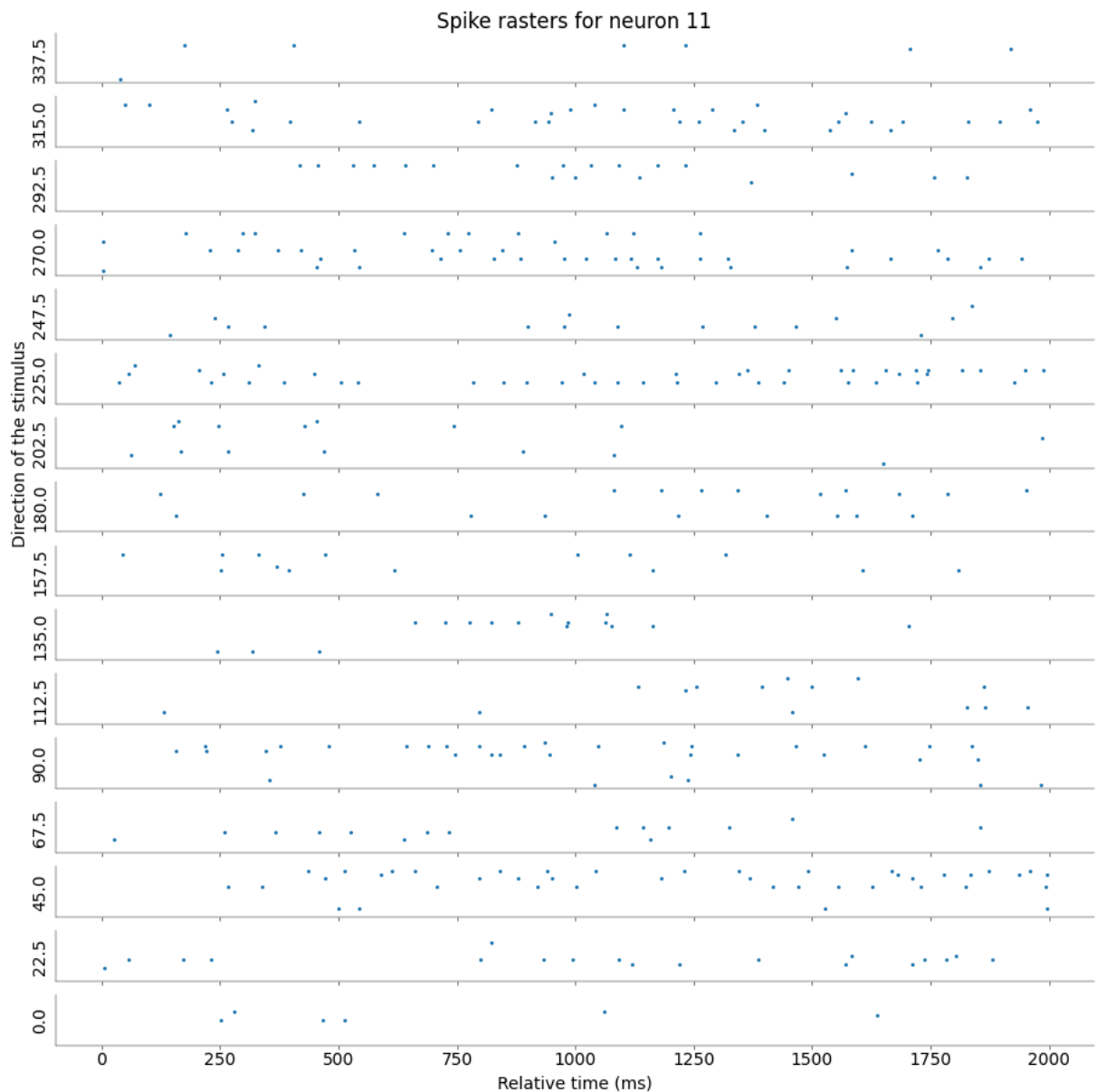
```



```
In [9]: # 2. orientation selective: neuron responds equally to both directions of the stimulus
        plotRaster(spikes, 28)
```



```
In [10]: # 3. neither  
plotRaster(spikes, 11)
```



Task 2: Plot spike density functions

Compute an estimate of the spike rate against time relative to stimulus onset.

There are two ways:

- Discretize time: Decide on a bin size, count the spikes in each bin and average across trials.
- Directly estimate the probability of spiking using a density estimator with specified kernel width.

For full points, the optimal kernel- or bin-width needs to be computed.

Implement one of them in the function `plotPSTH()`. If you don't use a dataframe you may need to change the interface of the function.

Grading: 4 pts

```
In [11]: # Get the optimal bin width using Shimazaki & Shinomoto's error function (Sl
def optimal_bin_width(spike_times, T, n_bins_range=range(10, 200)):
    costs = [] # Store error cost values for each candidate bin width
    bin_widths = [] # Store the corresponding bin widths

    # Try to concatenate all spike times across trials into one flat array
    try:
        all_spikes = np.concatenate(spike_times)
    except ValueError:
        # If no spikes present, return a fallback bin width (e.g., 100 ms bi
        return T / 20

    # Loop over a range of candidate bin counts
    for nb in n_bins_range:
        # Divide the time window T into 'nb' equal-width bins
        bins = np.linspace(0, T, nb + 1)

        # Count how many spikes fall into each bin
        counts, _ = np.histogram(all_spikes, bins=bins)

        # Compute average spike count per bin ( $\bar{k}$  in slide 15)
        k_bar = np.mean(counts)

        # Compute variance of spike counts across bins ( $v$  in slide 15)
        v = np.var(counts)

        # Compute bin width  $\Delta$ 
        delta = T / nb

        # cost function (Shimazaki & Shinomoto 2007):
        #  $C_n(\Delta) = (2 * \bar{k} - v) / \Delta^2$ 
        # This approximates the tradeoff between bias and variance
        cost = (2 * k_bar - v) / (delta**2)

        # Store results
        costs.append(cost)
        bin_widths.append(delta)

    # Return the bin width that minimizes the cost function
    return bin_widths[np.argmin(costs)]
```

```
In [12]: def plotPSTH(spikes: pd.DataFrame, neuron: int):
    """Plot PSTH for a single neuron sorted by condition

    Parameters
    -----

    spikes: pd.DataFrame
        Pandas DataFrame with columns
            Neuron | SpikeTimes | Dir | relTime | Trial | stimPeriod

    neuron: int
        Neuron ID
```

Note

this function does not return anything, it just creates a plot!
"""

```
# -----  
# Implement one of the spike rate estimates (3 pts)  
# -----  
  
# Get sorted list of unique stimulus directions  
dirs = sorted(spikes["Dir"].unique())  
stim_duration = 2000 # ms  
  
# Create subplots  
fig, axes = plt.subplots(len(dirs), 1, figsize=(10, 12), sharex=True)  
  
# Reverse the direction of subplot indexing to go bottom-up  
for i, direction in enumerate(sorted(dirs)):  
    ax = axes[len(dirs) - 1 - i] # Reverse order here  
  
    # Select spike data for this neuron and stimulus direction  
    # Only include spikes that occurred during the stimulus  
    condition_data = spikes[  
        (spikes["Neuron"] == neuron)  
        & (spikes["Dir"] == direction)  
        & (spikes["stimPeriod"])  
    ]  
    # Group spike times by trial  
    trials = condition_data["Trial"].unique()  
    spike_times_per_trial = [  
        condition_data[condition_data["Trial"] == t]["relTime"].values  
        for t in trials  
    ]  
    # Concatenate all spike times for histogram counting  
    # If no spikes are present, fall back to an empty array  
    all_spikes = (  
        np.concatenate(spike_times_per_trial)  
        if spike_times_per_trial and any(len(x) > 0 for x in spike_times_per_trial)  
        else np.array([])  
    )  
    # Handle empty bins  
    if all_spikes.size == 0:  
        ax.text(  
            0.5,  
            0.5,  
            "No spikes",  
            transform=ax.transAxes,  
            ha="center",  
            va="center",  
            fontsize=12,  
            color="red",  
        )  
    ax.set_ylabel(f"{int(direction)}°")
```

```

        ax.set_yticks([])
        continue

    # Estimate optimal bin width using error function (slide 15)
    optimal_width = optimal_bin_width(spike_times_per_trial, T=stim_duration)
    bins = np.arange(0, stim_duration + optimal_width, optimal_width)

    hist, _ = np.histogram(all_spikes, bins=bins)
    psth = hist / (len(trials) * optimal_width / 1000.0) # spikes/sec

    # -----
    # Plot the obtained spike rate estimates (1 pt)
    # -----
    ax.bar(bins[:-1], psth, width=optimal_width, align="edge", color="gray")
    ax.set_ylabel(f"{int(direction)}°")
    ax.set_ylim(0, np.max(psth) * 1.1)

    axes[-1].set_xlabel("Time (ms)")
    fig.suptitle(f"PSTH for Neuron {neuron}")
    plt.tight_layout()
    plt.show()

```

Plot the same 3 examples you selected in Task 1

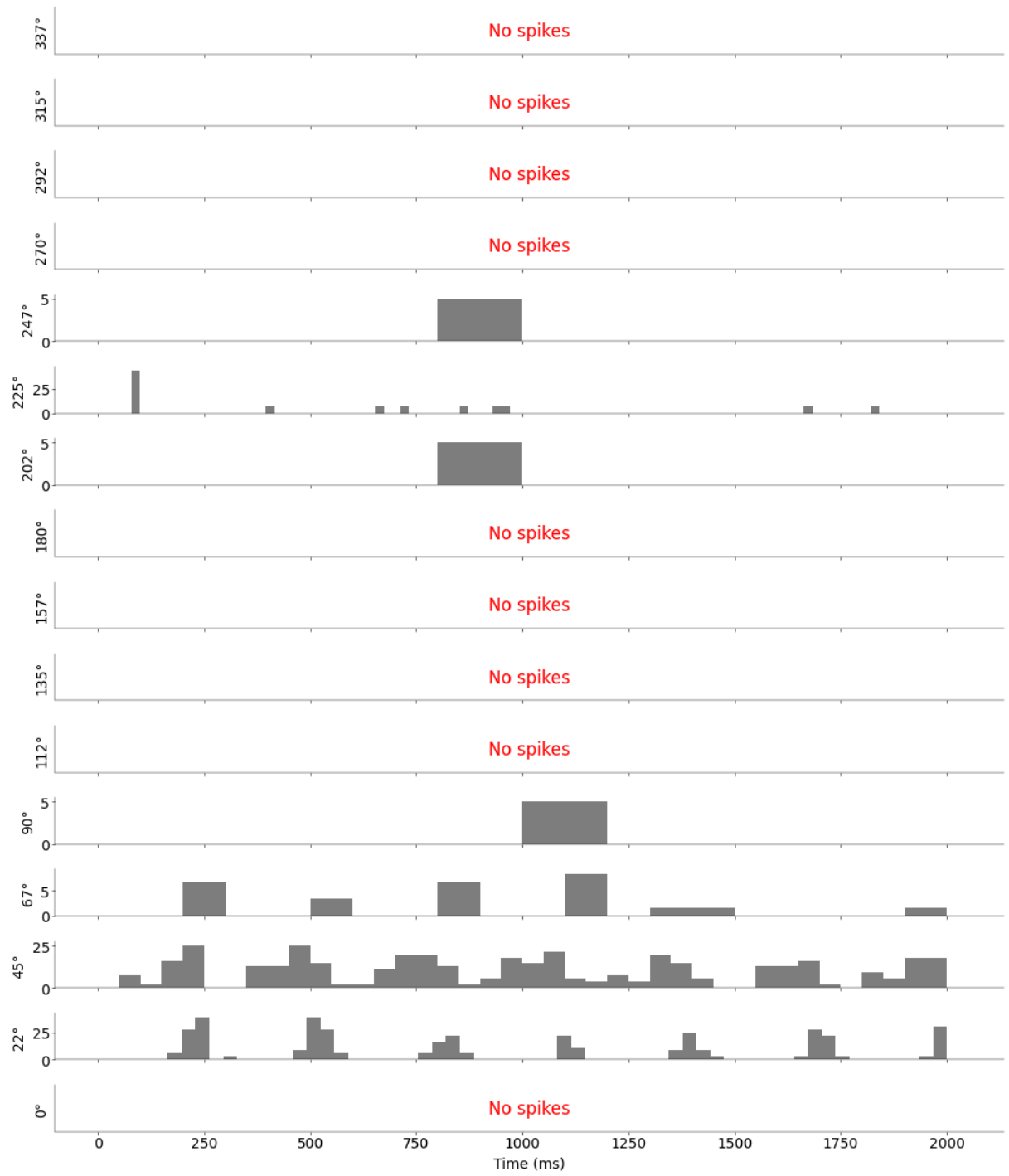
```

In [13]: plotPSTH(spikes, 27)
         plotPSTH(spikes, 28)
         plotPSTH(spikes, 11)

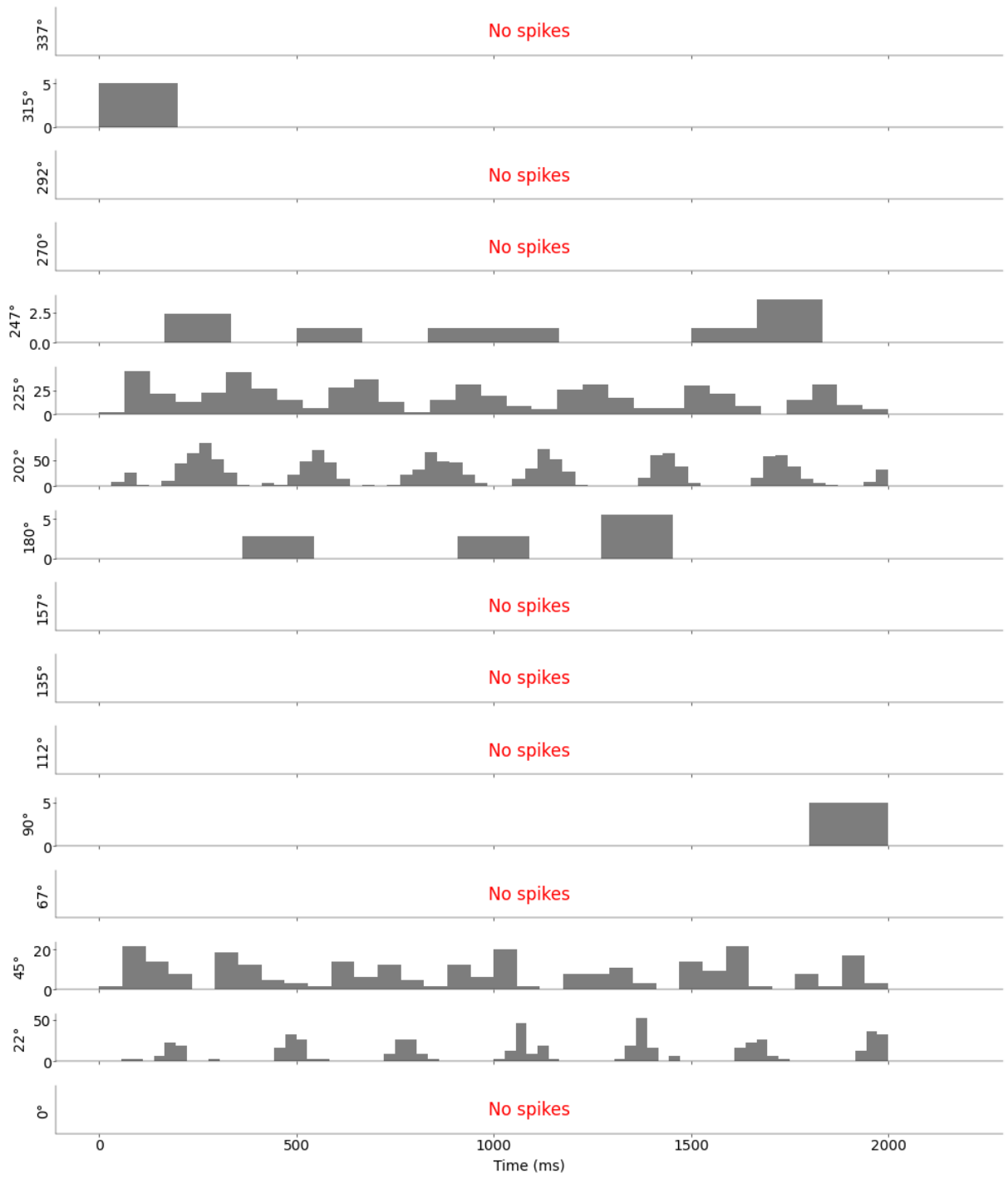
```

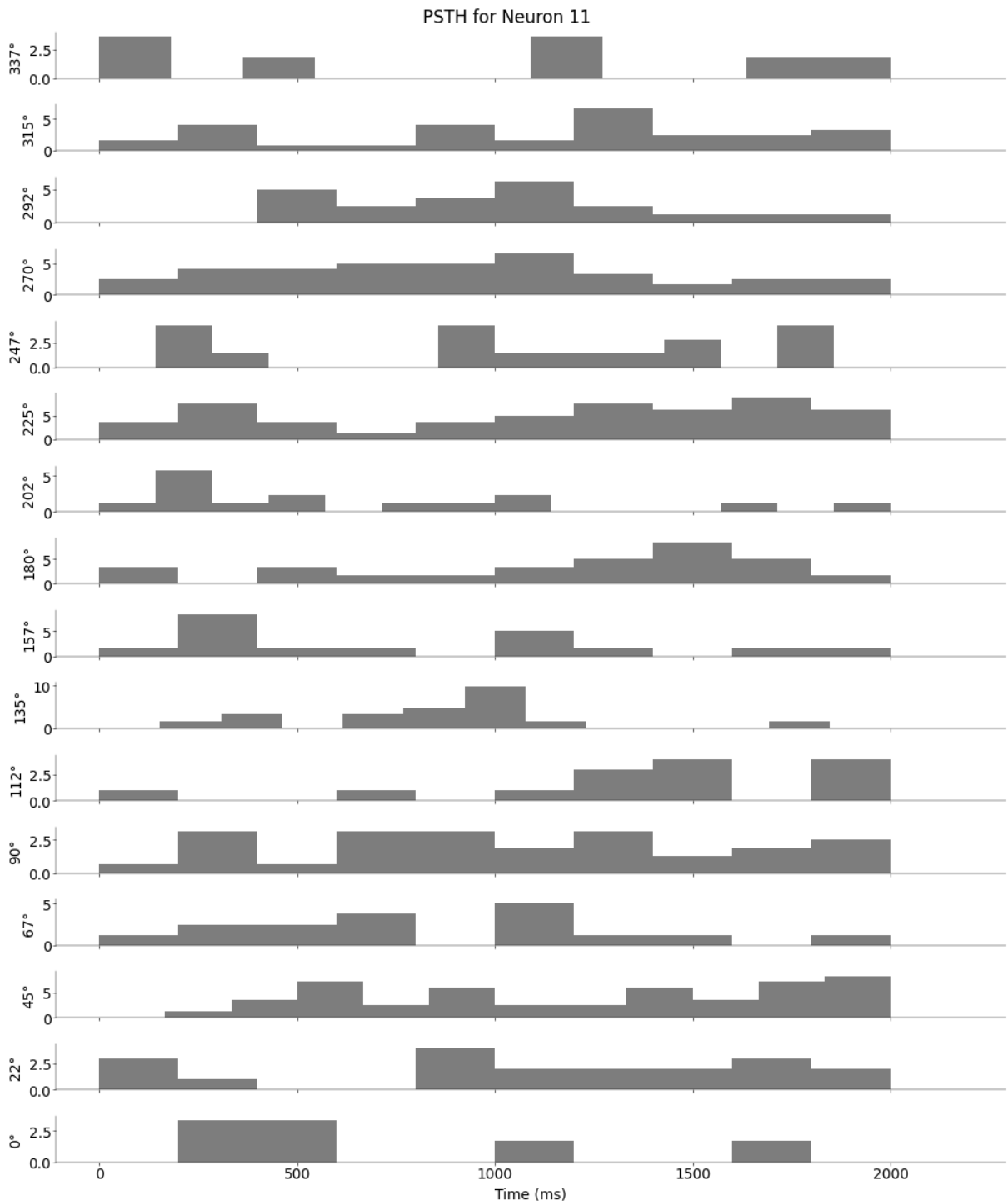
C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\79488289.py:88: UserWarning: The figure layout has changed to tight
 plt.tight_layout()

PSTH for Neuron 27



PSTH for Neuron 28





Task 3: Fit and plot tuning functions

The goal is to visualize the activity of each neuron as a function of stimulus direction. First, compute the spike counts of each neuron for each direction of motion and trial. The result should be a matrix $\mathbf{x}$, where x_{jk} represents the spike count of the j -th response to the k -th direction of motion (i.e. each column contains the spike counts for all trials with one direction of motion). If you used dataframes above, the `groupby()` function allows to implement this very

compactly. Make sure you don't lose trials with zero spikes though. Again, other implementations are completely fine.

Fit the tuning curve, i.e. the average spike count per direction, using a von Mises model. To capture the non-linearity and direction selectivity of the neurons, we will fit a modified von Mises function:

$$f(\theta) = \exp(\alpha + \kappa(\cos(2 * (\theta - \phi)) - 1) + \nu(\cos(\theta - \phi) - 1))$$

Here, θ is the stimulus direction. Implement the von Mises function in `vonMises()` and plot it to understand how to interpret its parameters ϕ , κ , ν , α . Perform a non-linear least squares fit using a package/function of your choice. Implement the fitting in `tuningCurve()`.

Plot the average number of spikes per direction, the spike counts from individual trials as well as your optimal fit.

Select two cells that show nice tuning to test your code.

Grading: 5 pts

```
In [14]: def vonMises(theta: np.ndarray, alpha: float, kappa: float, nu: float, phi: float) -> np.ndarray:
    """Evaluate the parametric von Mises tuning curve with parameters p at locations theta.

    Parameters
    -----
    theta: np.array, shape=(N, )
        Locations. The input unit is degree.

    alpha, kappa, nu, phi : float
        Function parameters

    Return
    -----
    f: np.array, shape=(N, )
        Tuning curve.
    """

    # -----
    # Implement the Mises model (0.5 pts)

    theta_rad = np.deg2rad(theta) # Convert to radians

    return np.exp(
        alpha + kappa * (np.cos(2 * (theta_rad - phi)) - 1) + nu * (np.cos(theta_rad - phi) - 1)
    )
```

Plot the von Mises function while varying the parameters systematically.

```
In [15]: # -----
# plot von Mises curves with varying parameters and explain what they do (2
```

```

fig, axes = plt.subplots(2, 2, figsize=(20, 8), sharex=True)
cmap = mpl.cm.get_cmap("plasma", 10)
fontsize_legend = 8
grid = np.linspace(0, 360, 100)

ax = axes[0, 0]
alphas = np.linspace(-1, 1, 10)
for i, alpha in enumerate(alphas):
    f = vonMises(grid, alpha, 0.5, 0.5, np.pi / 4)
    ax.plot(grid, f, linewidth=2, color=cmap(i), label=f"$\\alpha={alpha:.1f}$")
ax.set_ylabel("f( $\theta$ )")
ax.legend(prop={"size": fontsize_legend}, loc="upper right")
ax.set_title("Changing  $\\alpha$ ")

ax = axes[0, 1]
kappas = np.linspace(0, 1, 10)
for i, kappa in enumerate(kappas):
    f = vonMises(grid, 1, kappa, 0.5, np.pi / 4)
    ax.plot(grid, f, linewidth=2, color=cmap(i), label=f"$\\kappa={kappa:.1f}$")
ax.legend(prop={"size": fontsize_legend}, loc="upper right")
ax.set_title("Changing  $\\kappa$ ")

ax = axes[1, 0]
nus = np.linspace(0, 1, 10)
for i, nu in enumerate(nus):
    f = vonMises(grid, 1, 1, nu, np.pi / 4)
    ax.plot(grid, f, linewidth=2, color=cmap(i), label=f"$\\nu={nu:.1f}$")
ax.set_ylabel("f( $\theta$ )")
ax.legend(prop={"size": fontsize_legend}, loc="upper right")
ax.set_title("Changing  $\\nu$ ")
ax.set_xlabel("Direction (deg)")

ax = axes[1, 1]
phis = np.linspace(0, np.pi / 2, 10)
for i, phi in enumerate(phis):
    f = vonMises(grid, 1, 1, 0.5, phi)
    ax.plot(
        grid,
        f,
        linewidth=2,
        color=cmap(i),
        label=f"$\\phi={np.rad2deg(phi):.1f}^\\circ$",
    )
ax.set_xlabel("Direction (deg)")
ax.legend(prop={"size": fontsize_legend}, loc="upper right")
ax.set_title("Changing  $\\phi$ ")

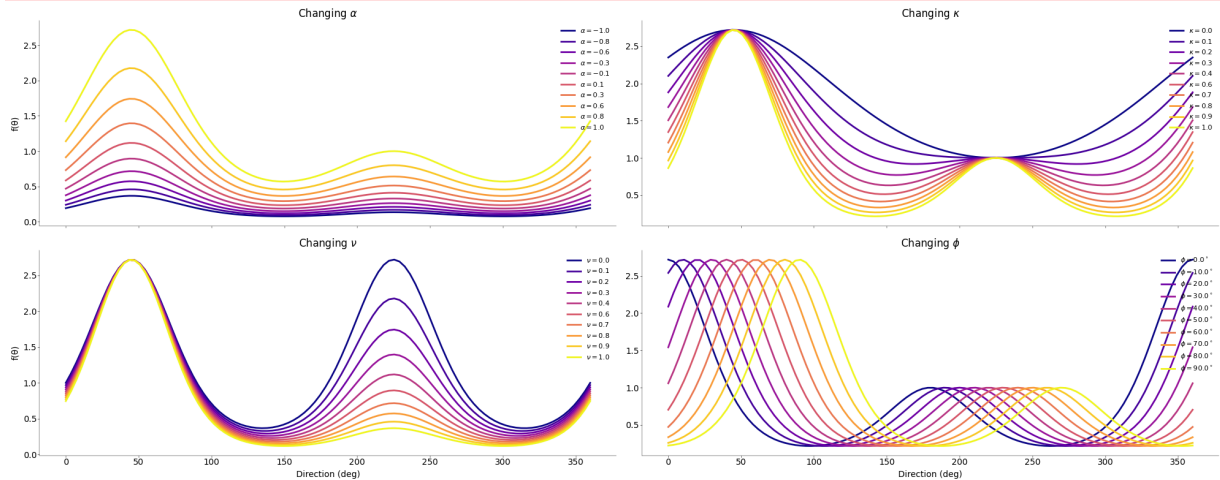
plt.tight_layout()
plt.show()
# -----

```



```
C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\1370736512.py:5: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
```

```
cmap = mpl.cm.get_cmap("plasma", 10)
C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\1370736512.py:55: UserWarning: The figure layout has changed to tight
plt.tight_layout()
```



Interpretation

α - captures baseline activity

κ - captures the selectivity of the function (steepness of tuning function), how much does response fall-off for nearby orientations.

ν - captures whether the function (for a given orientation) is selective only for one direction or both directions ($\theta, \theta + 180^\circ$)

ϕ - captures which orientation the function is selective to.

In []:

```
In [16]: from scipy.optimize import curve_fit

def tuningCurve(counts: np.ndarray, dirs: np.ndarray, show: bool = True) ->
    """Fit a von Mises tuning curve to the spike counts in count with direct

    Parameters
    -----

    counts: np.array, shape=(total_n_trials, )
        the spike count during the stimulation period

    dirs: np.array, shape=(total_n_trials, )
        the stimulus direction in degrees

    show: bool, default=True
```

Plot or not.

Return

```
-----  
p: np.array or list, (4,)  
    parameter vector of tuning curve function  
"""  
  
# -----  
# Compute the spike count matrix (0.5 pts)  
  
# Not sure why this is needed?  
  
# -----  
  
# -----  
# fit the von Mises tuning curve to the spike counts (0.5 pts)  
  
prob, _ = curve_fit(vonMises, dirs, counts)  
  
# -----  
if show == True:  
    # -----  
    # plot the data and fitted tuning curve (0.5 pts)  
    # -----  
    fig, ax = plt.subplots(figsize=(7, 5))  
    dirs_grid = np.linspace(0, 360, 100)  
    ax.plot(  
        dirs_grid,  
        vonMises(dirs_grid, *prob),  
        linewidth=2,  
        color="red",  
    )  
    ax.scatter(  
        dirs,  
        counts,  
        color="black",  
    )  
    ax.legend()  
    ax.set_xlabel("Orientation (°)")  
    ax.set_ylabel("Firing rate (sp/s)")  
    return ax  
else:  
    return prob
```

Plot tuning curve and fit for different neurons. Good candidates to check are 28, 29 or 37.

```
In [17]: def get_data(spikes, neuron):  
    spk_by_dir = (  
        spikes[spikes["Neuron"] == neuron]  
        .groupby(["Dir", "Trial"])["stimPeriod"]  
        .sum()  
        .astype(int)
```

```

        .reset_index()
    )

    dirs = spk_by_dir["Dir"].values
    counts = spk_by_dir["stimPeriod"].values

    # because we count spikes only when they are present, some zero entries
    for i, Dir in enumerate(np.unique(spikes["Dir"])):
        m = nTrials - np.sum(dirs == Dir)
        if m > 0:
            dirs = np.concatenate((dirs, np.ones(m) * Dir))
            counts = np.concatenate((counts, np.zeros(m)))

    idx = np.argsort(dirs)
    dirs_sorted = dirs[idx] # sorted dirs
    counts_sorted = counts[idx]

    return dirs_sorted, counts_sorted

```

```

In [18]: # -----
# plot the average number of spikes per direction, the spike
# counts from individual trials as well as your optimal fit
# for different neurons (0.5 pts)

for i_neuron in [28, 29, 37]:
    dirs, counts = get_data(spikes, i_neuron)
    ax = tuningCurve(counts, dirs, show=True)
    ax.set_title(f"Tuning curve - neuron {i_neuron}")

# -----

```

C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\682077167.py:56: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
ax.legend()
```

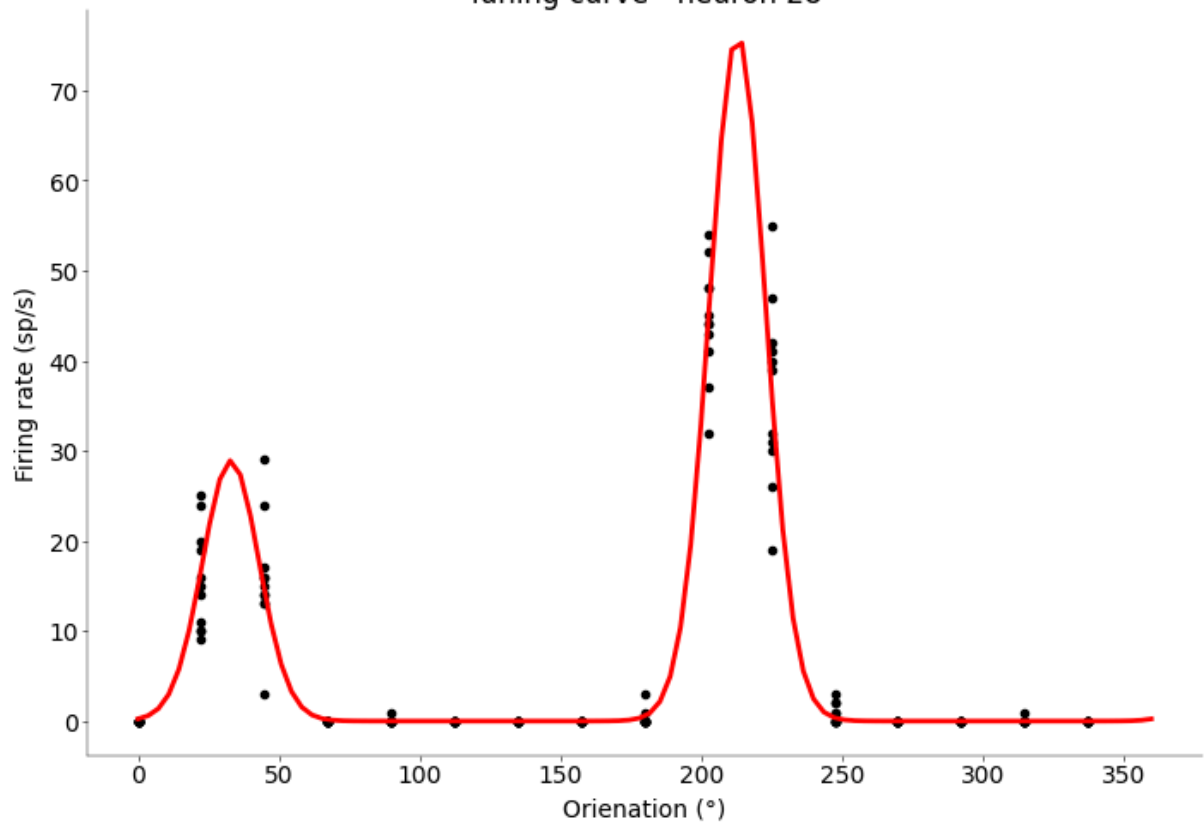
C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\682077167.py:56: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
ax.legend()
```

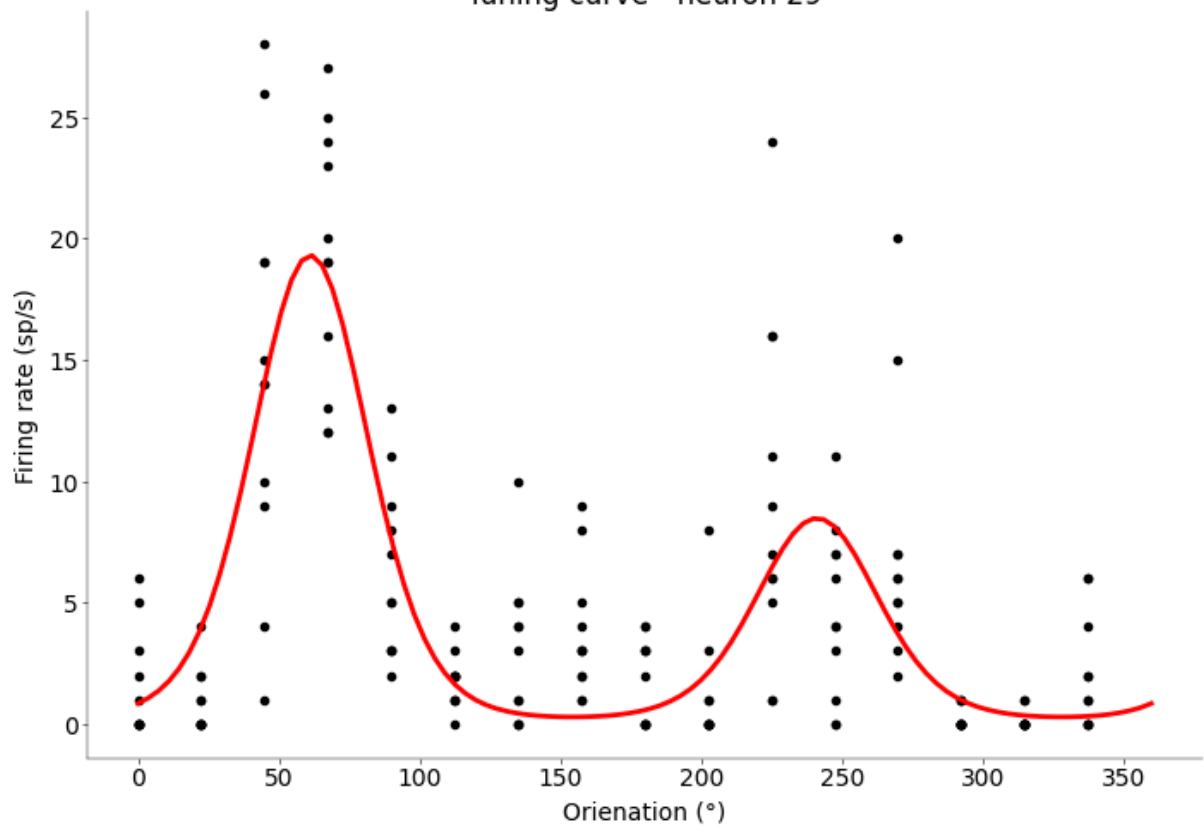
C:\Users\aksel\AppData\Local\Temp\ipykernel_32364\682077167.py:56: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

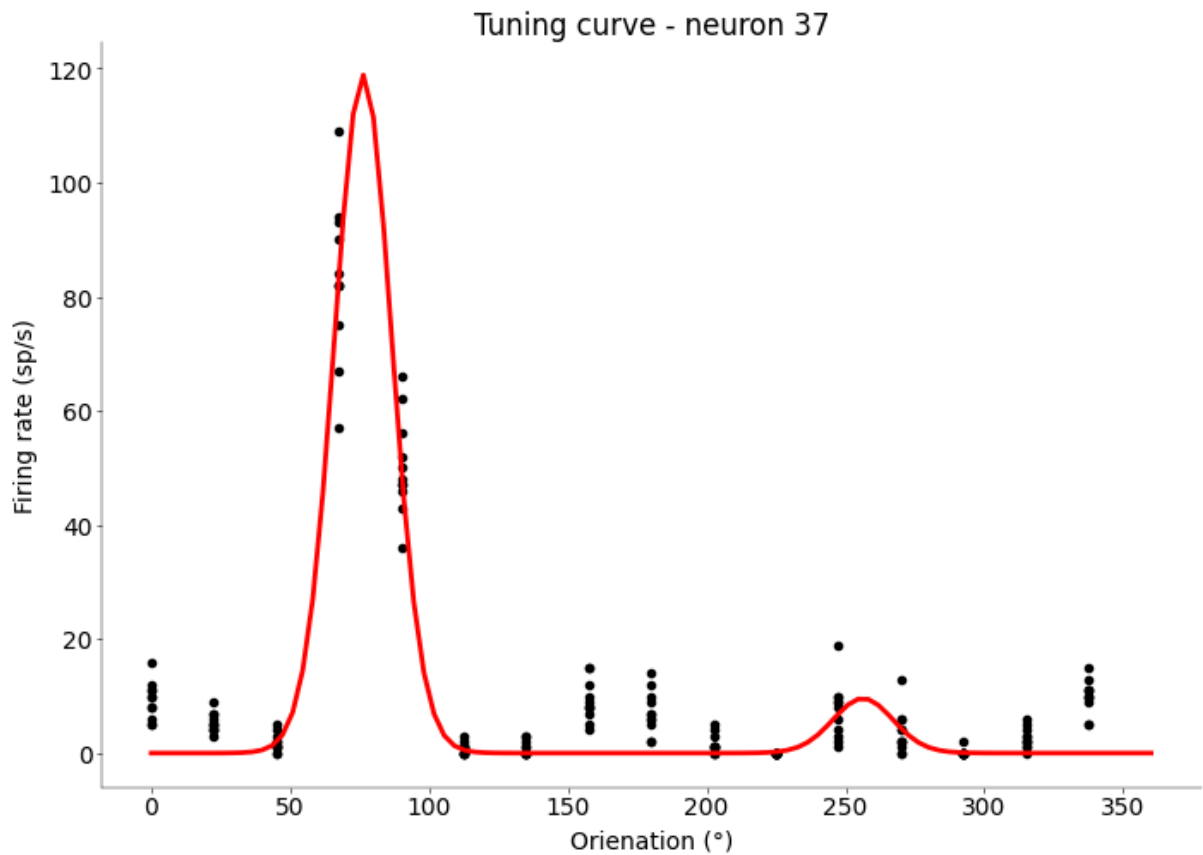
```
ax.legend()
```

Tuning curve - neuron 28



Tuning curve - neuron 29





Task 4: Permutation test for direction tuning

Implement a permutation test to quantitatively assess whether a neuron is direction/orientation selective. To do so, project the vector of average spike counts, $m_k = \frac{1}{N} \sum_j x_{jk}$ on a complex exponential with two cycles, $v_k = \exp(\psi i \theta_k)$, where θ_k is the k -th direction of motion in radians and $\psi \in 1, 2$ is the fourier component to test (1: direction, 2: orientation). Denote the projection by $q = m^T v$. The magnitude $|q|$ tells you how much power there is in the ψ -th fourier component.

Estimate the distribution of $|q|$ under the null hypothesis that the neuron fires randomly across directions by running 1000 iterations where you repeat the same calculation as above but on a random permutation of the trials (that is, randomly shuffle the entries in the spike count matrix x). The fraction of iterations for which you obtain a value more extreme than what you observed in the data is your p-value. Implement this procedure in the function `testTuning()`.

Illustrate the test procedure for one of the cells from above. Plot the sampling distribution of $|q|$ and indicate the value observed in the real data in your plot.

How many cells are tuned at $p < 0.01$?

Grading: 3 pts

```
In [37]: def testTuning(
    counts: np.ndarray,
    dirs: np.ndarray,
    psi: int = 1,
    niters: int = 1000,
    show: bool = False,
    random_seed: int = 2046,
) -> Tuple[float, float, np.ndarray]:
    """Plot the data if show is True, otherwise just return the fit.

    Parameters
    -----

    counts: np.array, shape=(total_n_trials, )
        the spike count during the stimulation period

    dirs: np.array, shape=(total_n_trials, )
        the stimulus direction in degrees

    psi: int
        fourier component to test (1 = direction, 2 = orientation)

    niters: int
        Number of iterations / permutation

    show: bool
        Plot or not.

    random_seed: int
        Random seed for reproducibility.

    Returns
    -----
    p: float
        p-value
    q: float
        magnitude of second Fourier component

    qdistr: np.array
        sampling distribution of |q| under the null hypothesis

    """

    # -----
    # calculate m, nu and q (0.5 pts)
    thetas = np.deg2rad(dirs)

    nu = np.exp(1j * psi * thetas)

    q = np.mean(counts * nu)
    q_obs = np.abs(q)

    # -----
```

```

# -----
# Estimate the distribution of q under the H0 and obtain the p value (1
# -----
# ensure reproducibility using a random number generator
# hint: access random functions of this generator
rng = np.random.default_rng(random_seed)

qdistr = np.zeros(niters)

for i in range(niters):
    permuted_counts = rng.permutation(counts)
    q_perm = np.mean(permuted_counts * nu)
    qdistr[i] = np.abs(q_perm)

p = np.mean(qdistr >= np.abs(q))

if show:
    fig = plt.figure(figsize=(6, 4))
    ax = sns.histplot(qdistr, kde=True, label="Null distribution")
    ax.set_xlabel("|q|")

    ax.axvline(
        x=np.abs(q),
        color="red",
        linestyle="--",
        label=f"q = {np.abs(q):.2f} (p ~ {p:.3f})",
    )

    plt.legend(fontsize=12)

    return ax
else:
    return p, np.abs(q), qdistr

```

Show null distribution for the example cell:

```

In [38]: # -----
# Plot null distributions for example cells 28 & 29. (1 pt)

neuron = 28
dirs, counts = get_data(spikes, neuron)
ax = testTuning(counts, dirs, 1, show=True)
ax.set_title(f"Neuron {neuron} - direction selectivity")

ax = testTuning(counts, dirs, 2, show=True)
ax.set_title(f"Neuron {neuron} - orientation selectivity")

neuron = 29
dirs, counts = get_data(spikes, neuron)
ax = testTuning(counts, dirs, 1, show=True)
ax.set_title(f"Neuron {neuron} - direction selectivity")

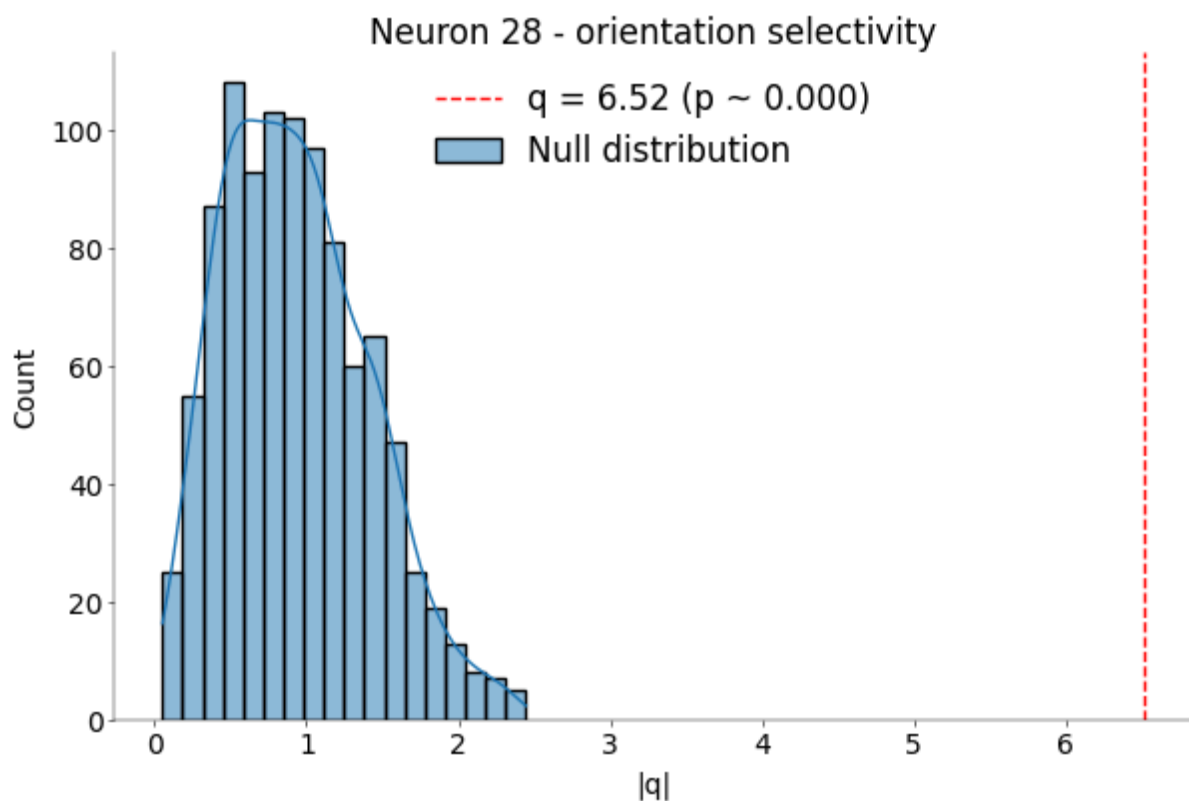
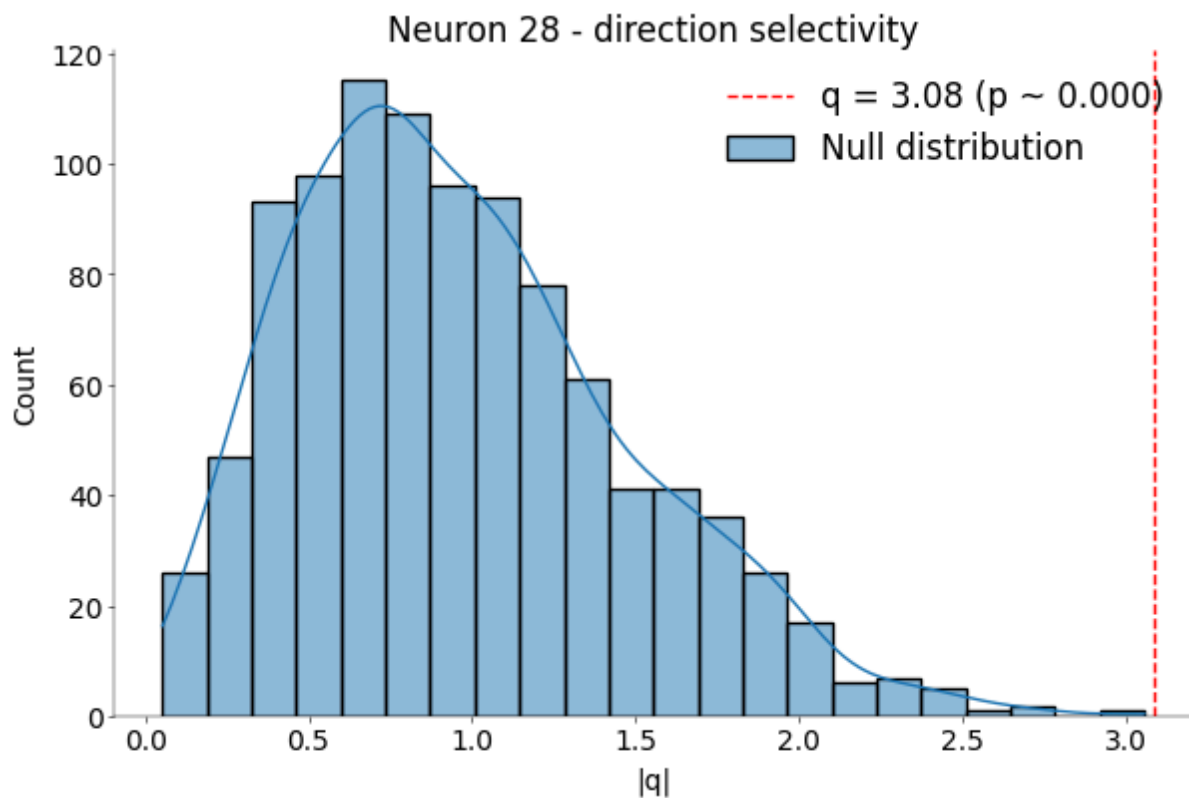
ax = testTuning(counts, dirs, 2, show=True)

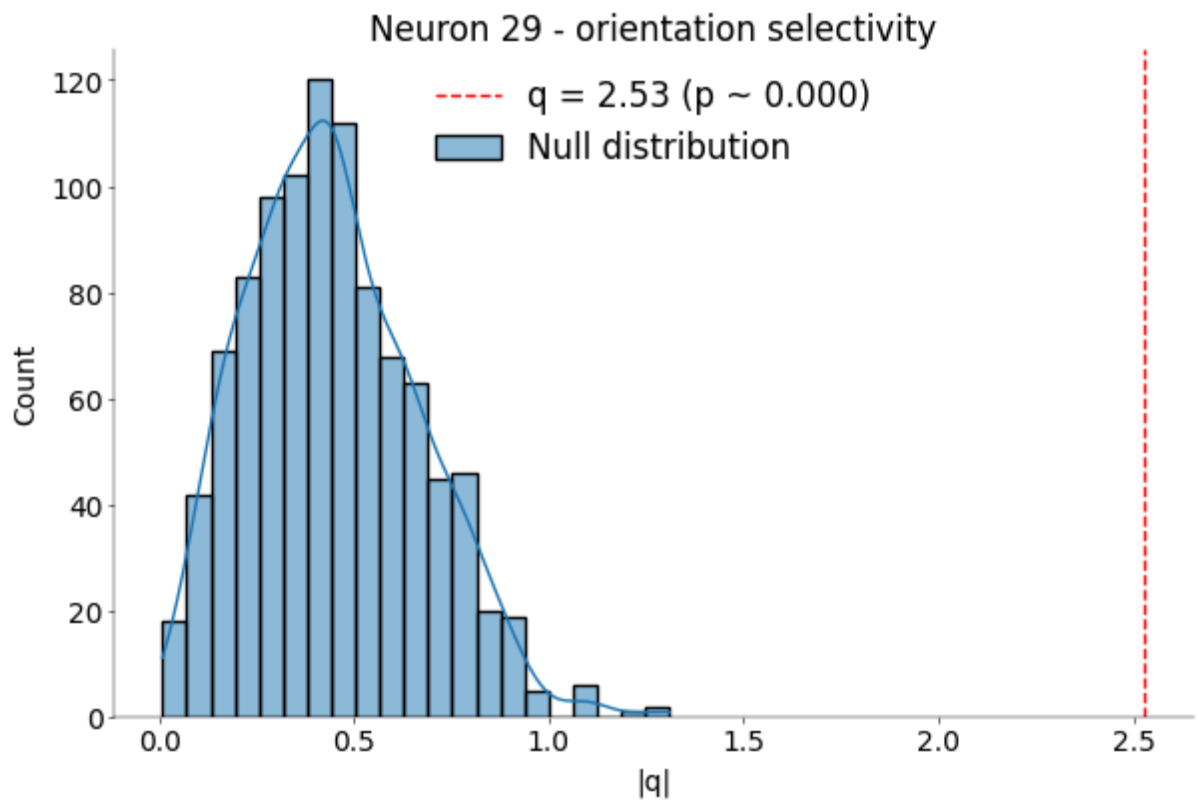
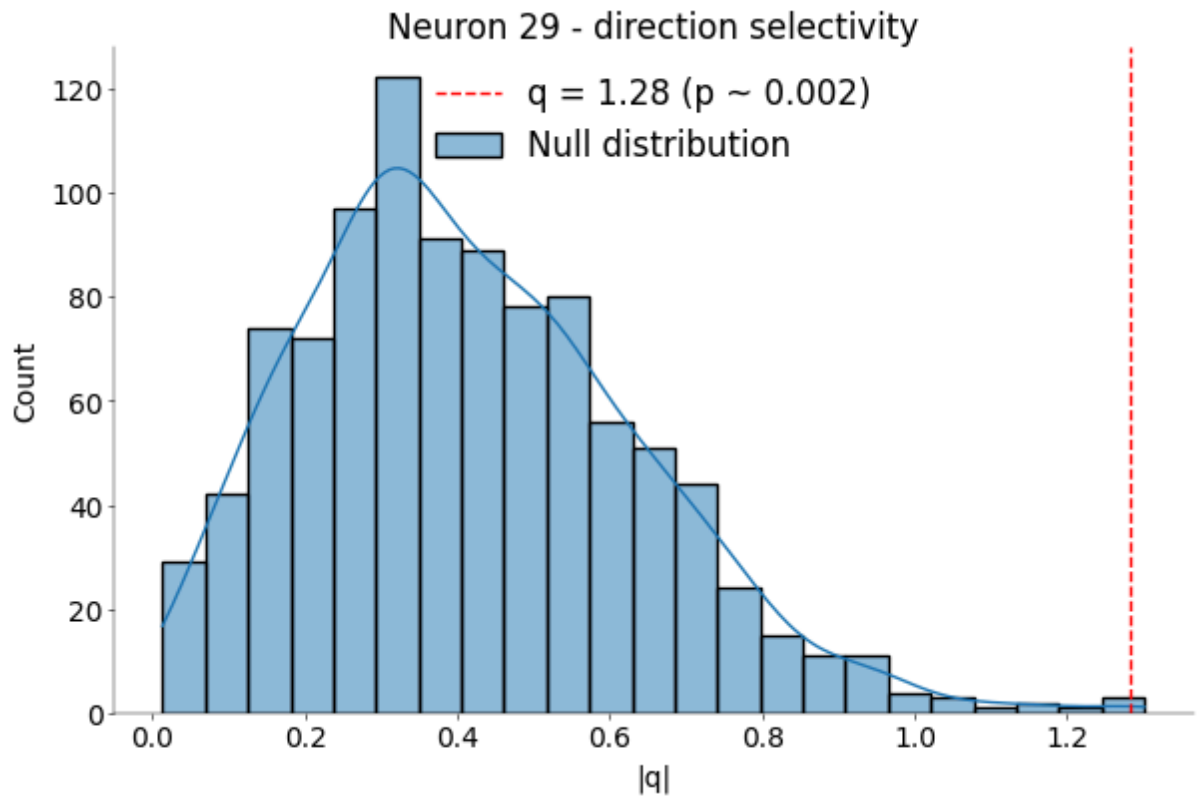
```

```
ax.set_title(f"Neuron {neuron} - orientation selectivity")
```

```
# -----
```

```
Out[38]: Text(0.5, 1.0, 'Neuron 29 - orientation selectivity')
```





Test all cells for orientation and direction tuning

```
In [25]: # -----
# Test all cells for orientation / direction tuning.
# Which ones are selective? (0.5 pts)
# -----
```

```

nNeurons = len(spikes["Neuron"].unique())

p_dir = np.zeros(nNeurons)
p_ori = np.zeros(nNeurons)

for n in range(nNeurons):
    dirs, counts = get_data(spikes, n)
    p_dir[n], q_dir, _ = testTuning(counts, dirs, psi=1, show=False)
    p_ori[n], q_ori, _ = testTuning(counts, dirs, psi=2, show=False)

```

Number of direction tuned neurons:

```

In [32]: alpha = 0.01

s = np.sum(p_dir < alpha)
print("Number of direction tuned neurons:", s)
print("Neurons:", np.where(p_dir < alpha)[0] + 1)

```

Number of direction tuned neurons: 12
 Neurons: [14 21 25 26 28 29 30 32 33 38 39 41]

Number of orientation tuned neurons:

```

In [33]: s = np.sum(p_ori < alpha)
print("Number of orientation tuned neurons:", s)
print("Neurons:", np.where(p_ori < alpha)[0] + 1)

```

Number of orientation tuned neurons: 33
 Neurons: [3 4 7 8 9 11 13 14 15 16 17 18 19 21 22 23 24 25 26 27 28 29
 30 31
 32 33 34 35 37 38 39 40 41]

In []: